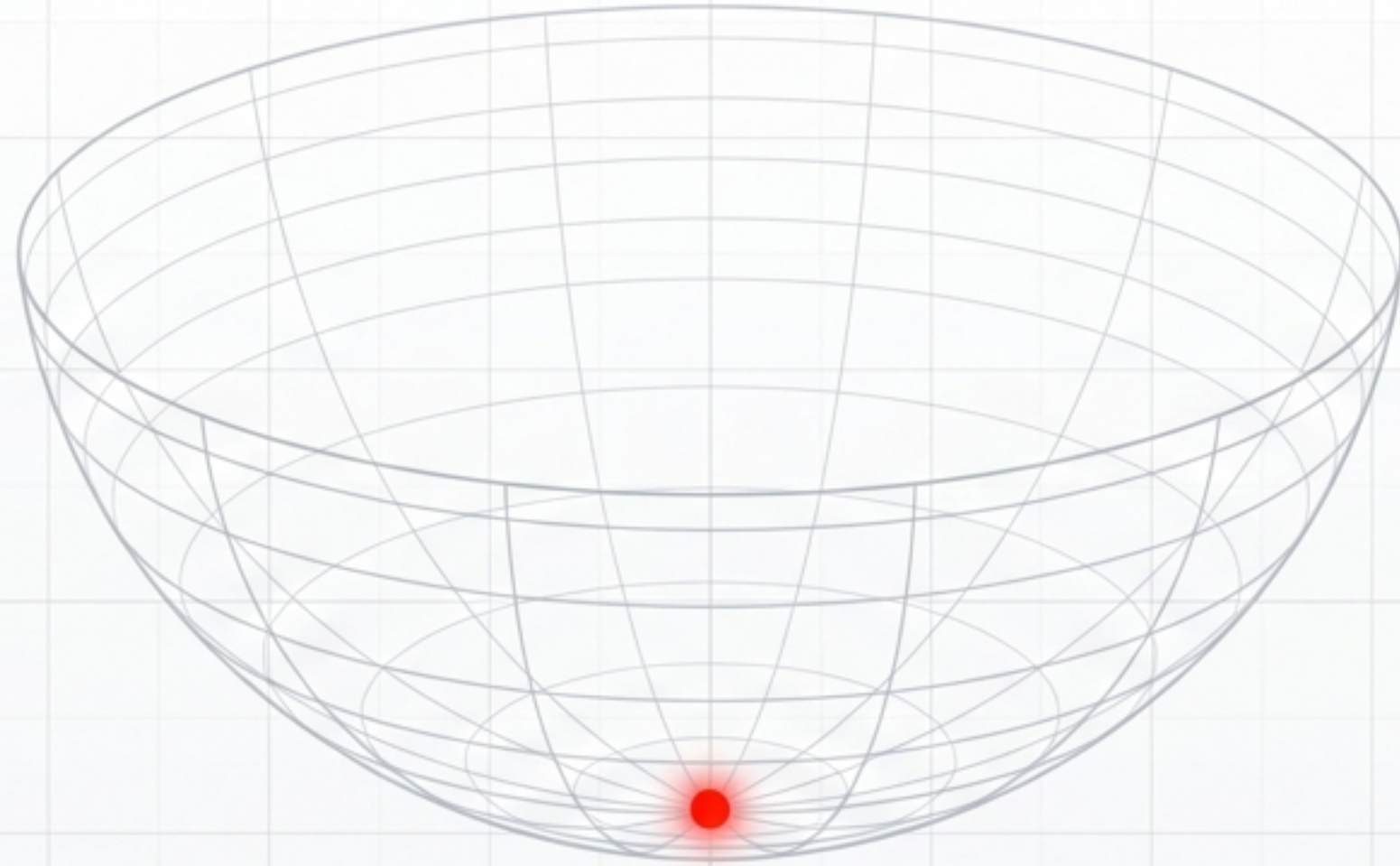




Mapping the Topography of Linear Regression

Cost Function Intuition, Code Refactoring, and Visual Insights.

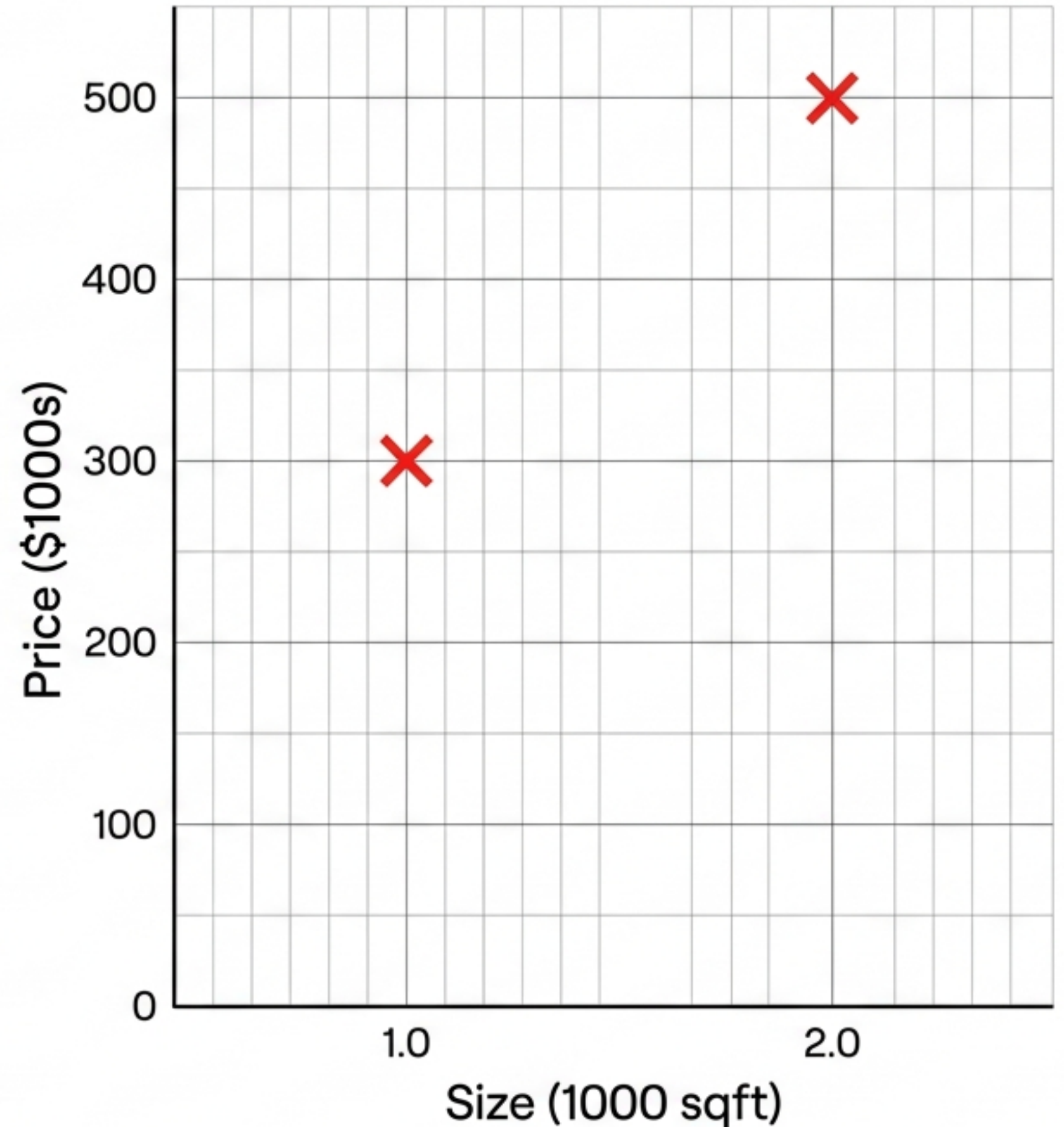
Defining the Baseline Prediction Model

The objective is to predict housing prices based on square footage using a simple, univariable linear model.



$$f_{w,b}(x) = wx + b$$

- | | | |
|-----|--|--|
| x | | Input feature
(house size in 1000 sqft) |
| w | | The slope (weight) |
| b | | The intercept (bias) |



Deconstructing the Cost Function Engine

The Averaging Factor:

Averages the error across all m examples (the $1/2$ simplifies the subsequent derivative for gradient descent).

The Penalty:

Squaring the difference ensures all errors are positive and heavily punishes massive deviations.

$$J(w, b) = \frac{1}{2m} \sum_{i=0}^{m-1} (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

The Prediction:

The model's calculated output for example i .

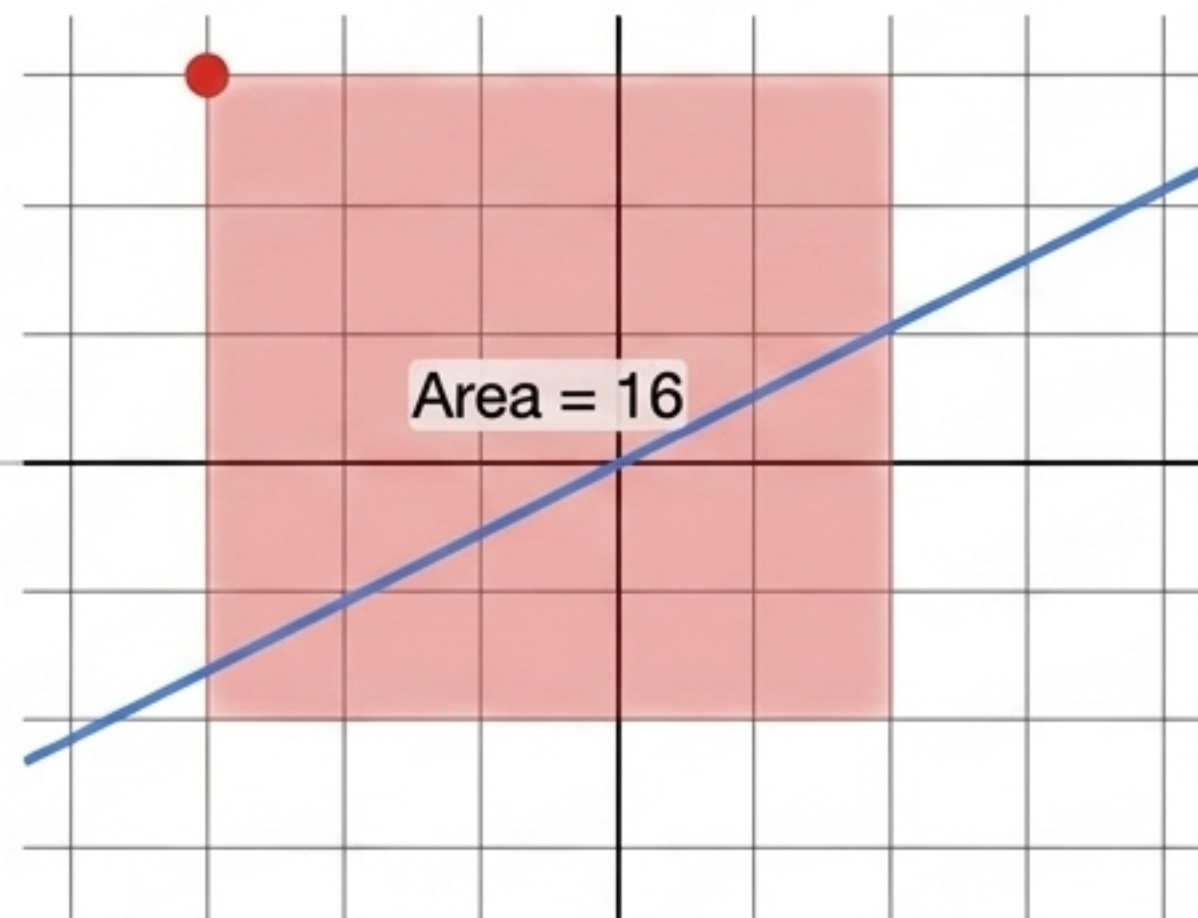
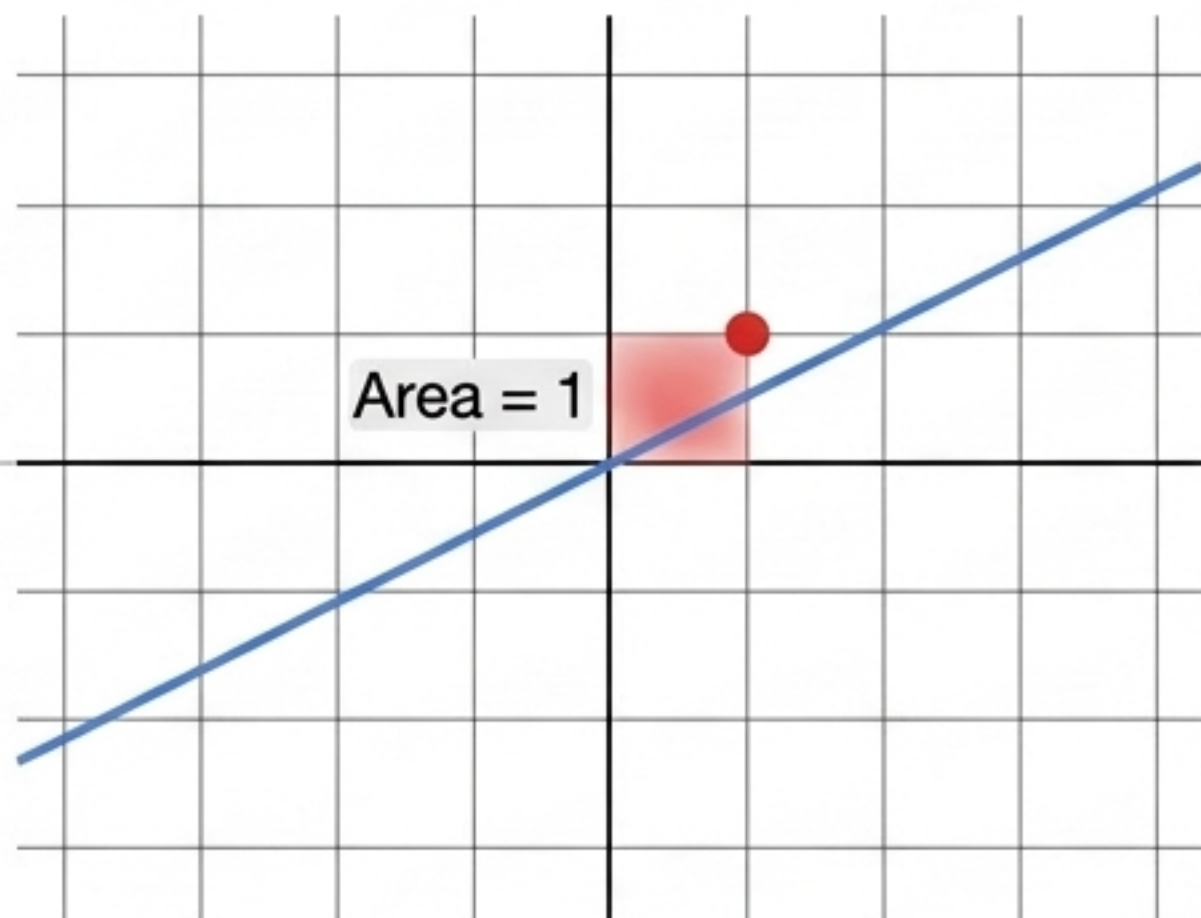
The Reality:

The actual, true target label from the dataset.

$J(w, b)$ is a single number summarizing overall fit quality. Lower is better. Zero is perfect.

The Mathematical Logic of the Squared Penalty

Why use $(f - y)^2$ instead of absolute difference?



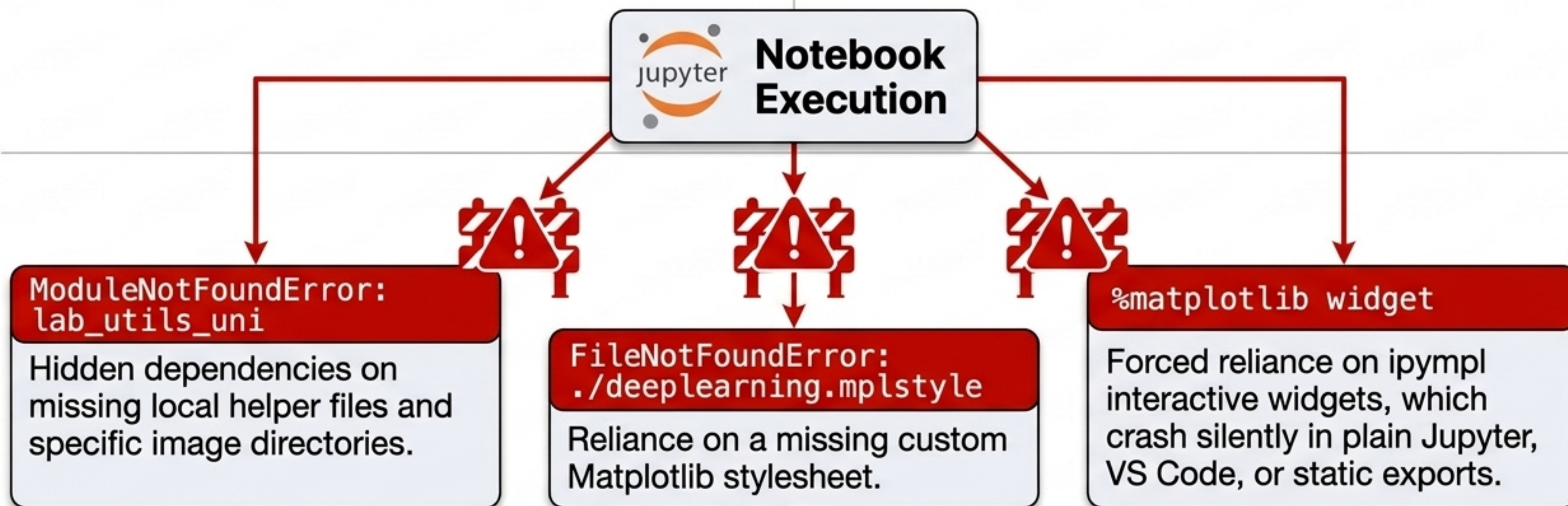
Guarantees **non-negative** error values, preventing positive and negative errors from canceling each other out.

Disproportionately penalizes outliers. A mistake of 4 units is punished 16 times harder than a mistake of 1 unit.

Produces a **smooth**, continuously **differentiable** surface critical for calculus-based optimization.

Diagnosing Fragility in the Original Laboratory

The algorithmic implementation of `compute_cost()` was mathematically flawless. However, the notebook's execution environment was highly brittle outside the specific course ecosystem.

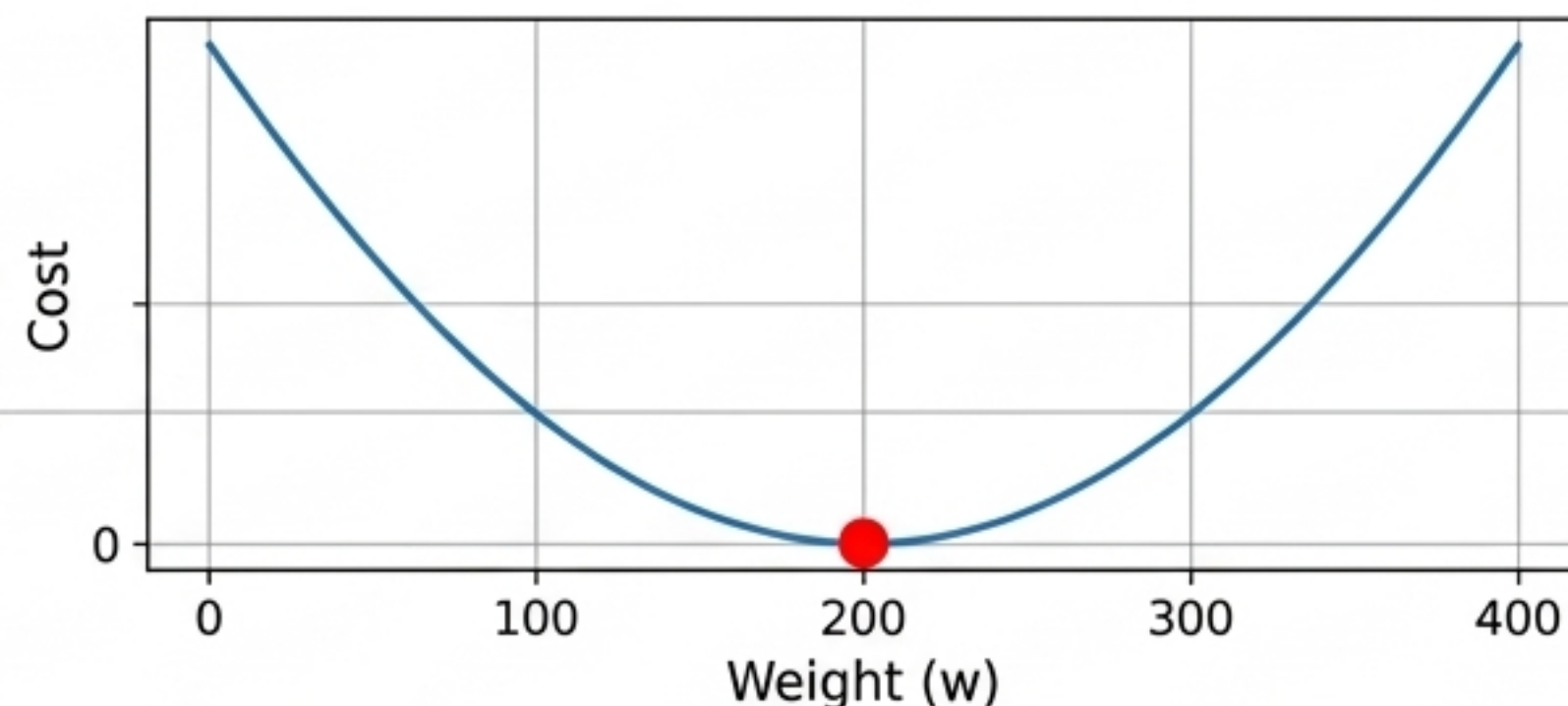
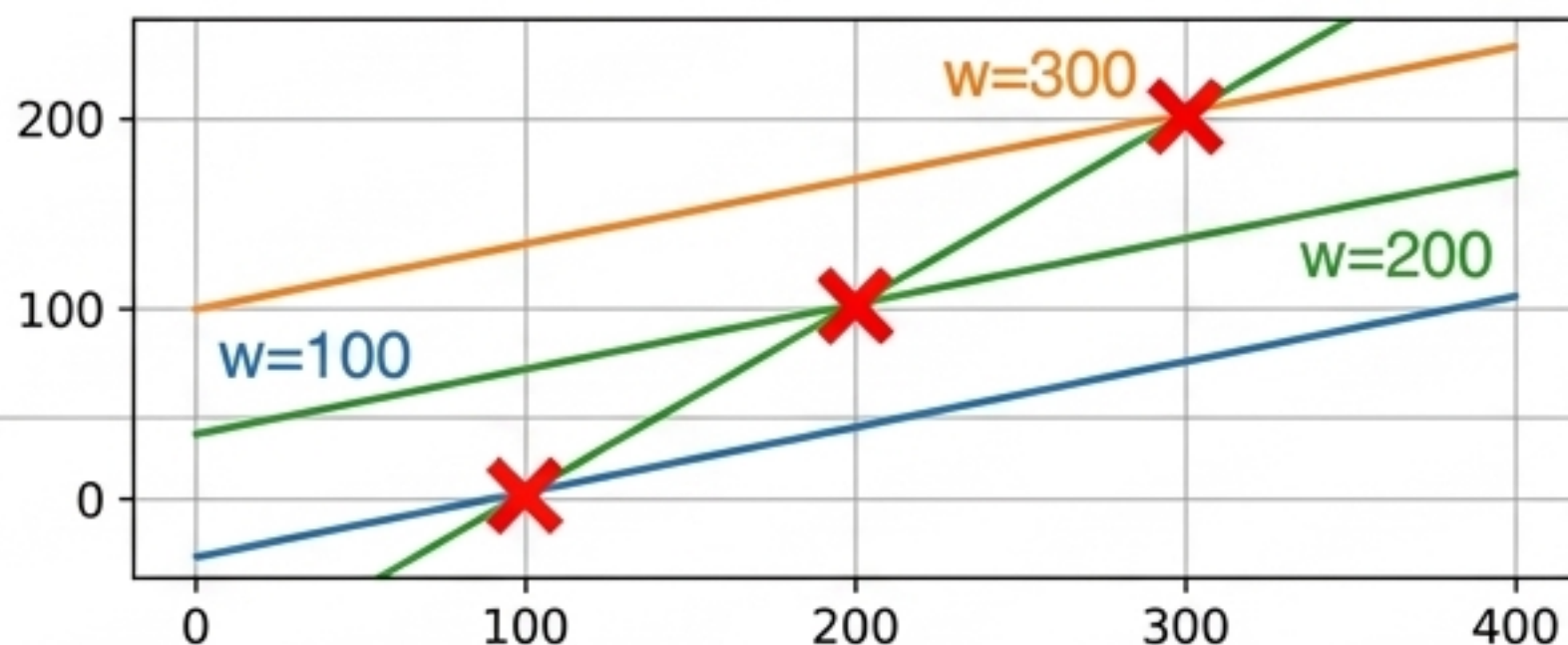


Engineering a Robust, Self-Contained Execution

	Original State (Fragile)	Refactored State (Robust)
External Dependencies	Required missing <code>lab_utils_uni.py</code> and local image files. ⚙️	Replaced with self-contained, inline plotting functions using only standard NumPy and Matplotlib. ✓
Styling Engine	Depended on absent <code>./deeplearning.mplstyle</code> . ⚙️	Leverages Matplotlib defaults with clean, lightweight alpha grids (<code>plt.rcParams['grid.alpha'] = 0.3</code>). ✓
Execution Reliability & Interactivity	Hardcoded <code>%matplotlib</code> widget caused environments to crash. ✖️	Implemented try/except fallback logic for ipywidgets. Gracefully downgrades to static inline plots if a live front-end is unavailable. ✖️

1D Topography: Isolating the Weight Parameter

By locking the bias parameter ($b=100$), we can sweep the weight (w) from 0 to 400 to observe the direct impact on model accuracy.



The Perfect Fit

At $w=200$, the model's line perfectly intersects both training points.

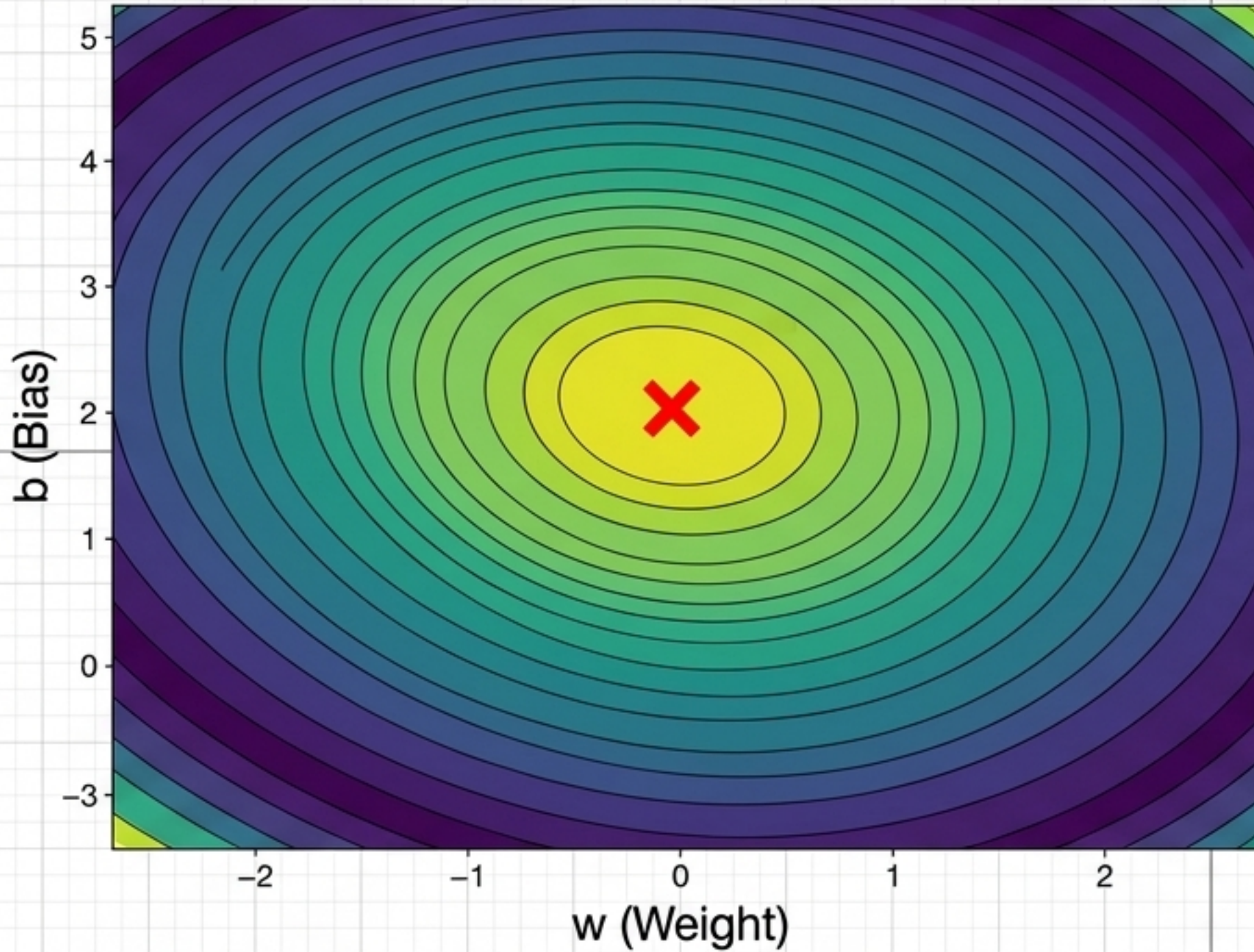
Zero Cost

Consequently, the cost curve bottoms out at exactly 0.

The Parabolic Shape

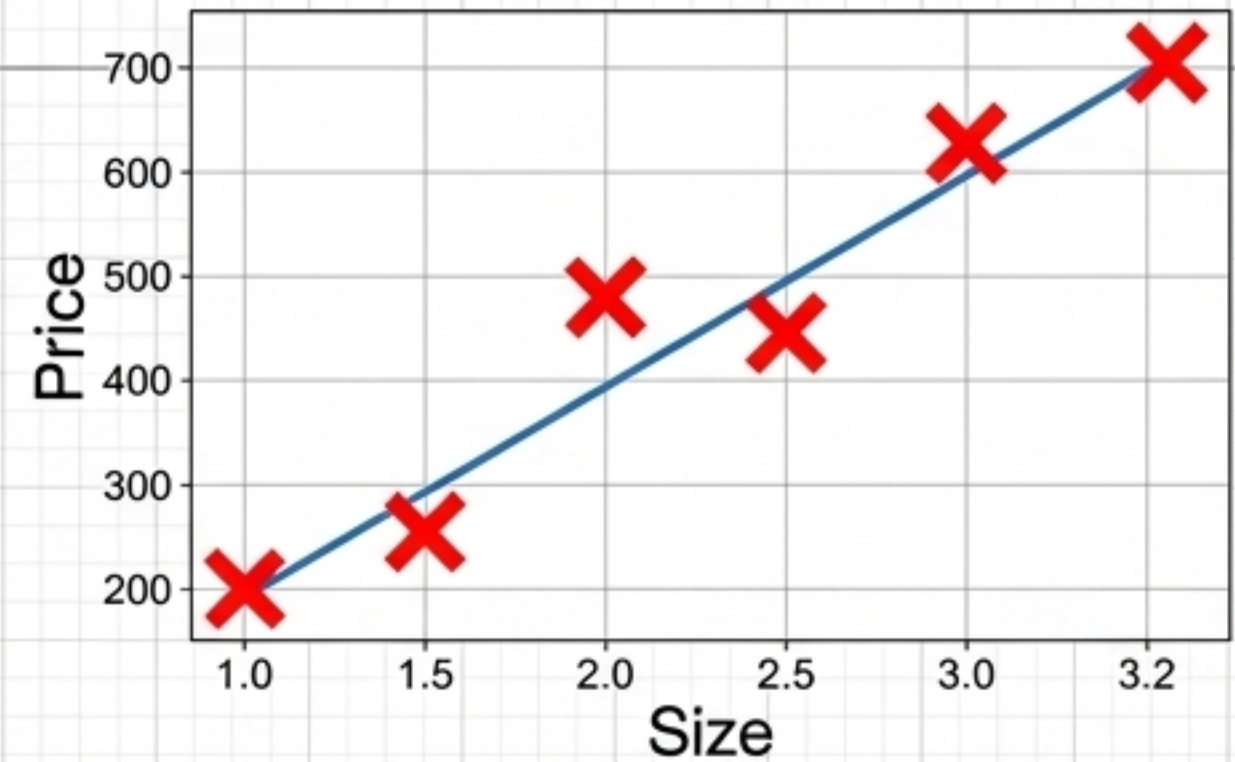
The squared error creates steep walls; cost rises exponentially the further w drifts from the optimum.

2D Topography: Navigating the Contour Basin



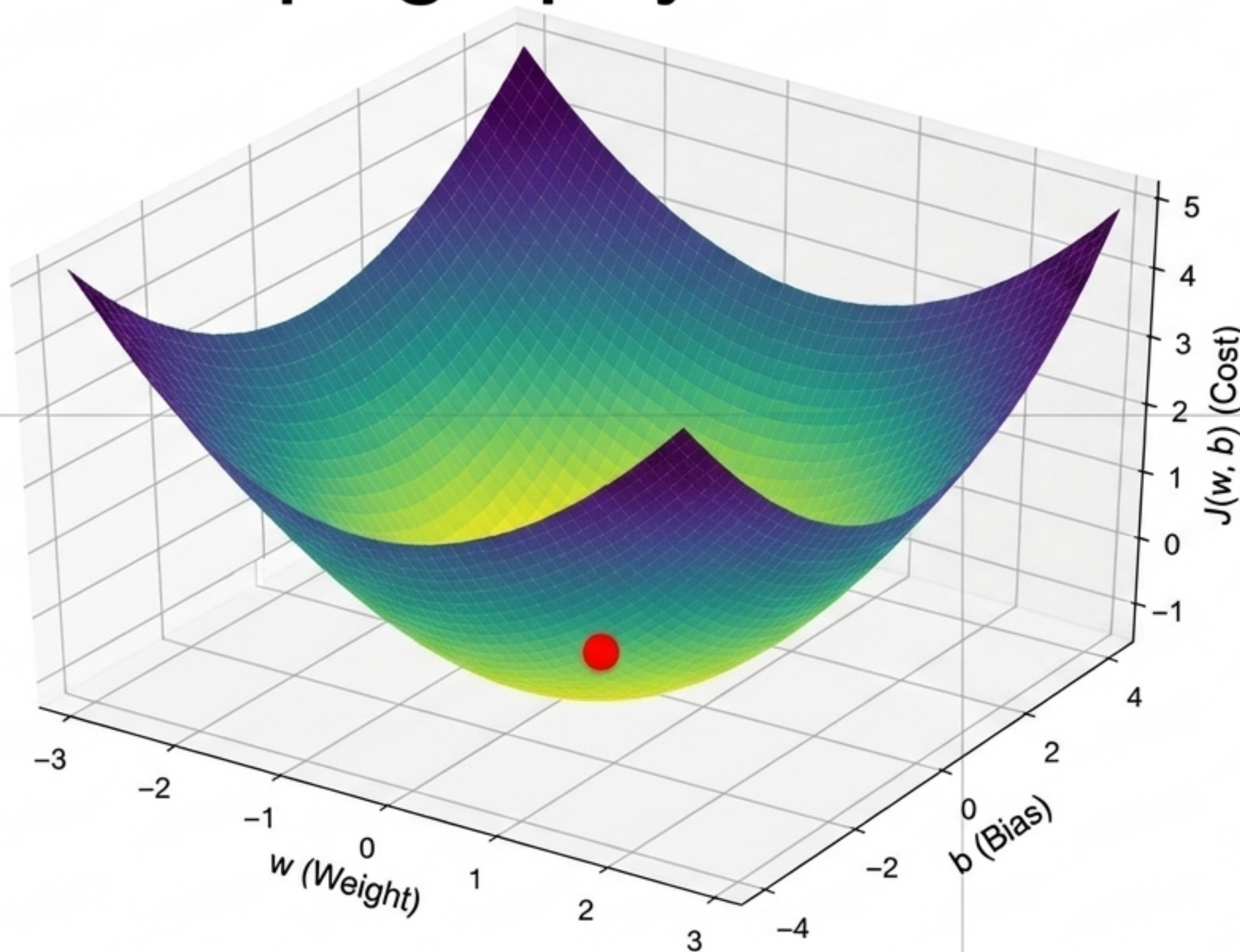
Real-world data scatters. With the expanded dataset, a mathematically perfect 0 cost is no longer possible.

Size:	1.0	1.7	2.0	2.5	3.0	3.2
Price:	250	300	480	430	630	730



Evaluating $J(w, b)$ over a complete grid reveals a single basin. The contour rings shrink toward the absolute lowest cost at $w \approx 209$, $b \approx 2.4$. Moving away results in higher error.

3D Topography: The Convex Cost Surface



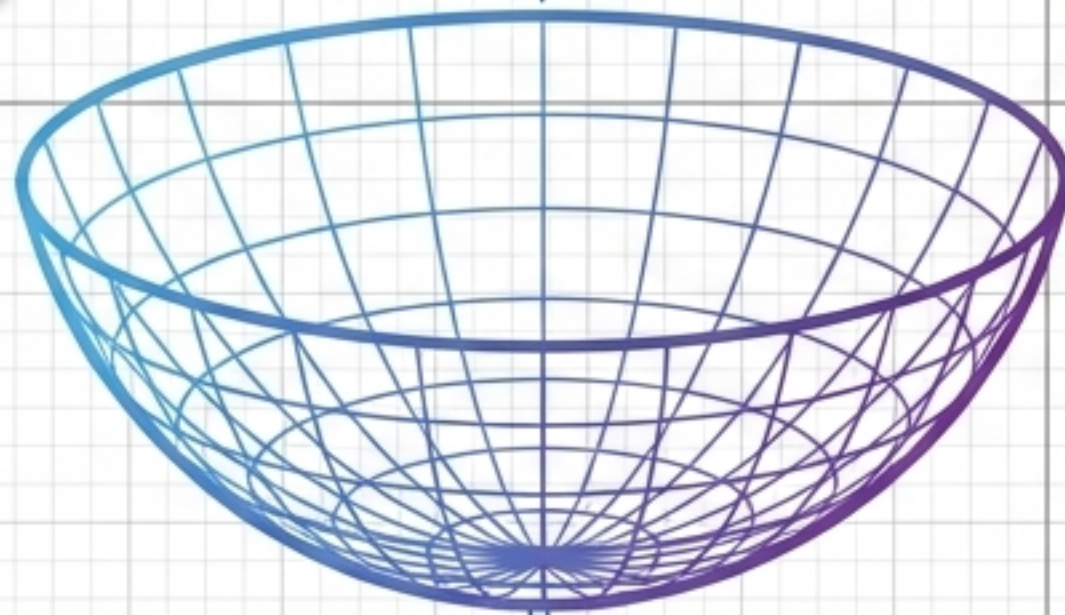
Plotting the grid of $J(w, b)$ values in three dimensions immediately reveals the structural shape behind linear regression optimization.

The error topography is entirely smooth and bowl-shaped. From the highest peaks down to the lowest red dot ($w \approx 2.09$, $b \approx 2.4$), there are no jagged cliffs or hidden valleys.

Synthesis: The Guarantee of Convexity

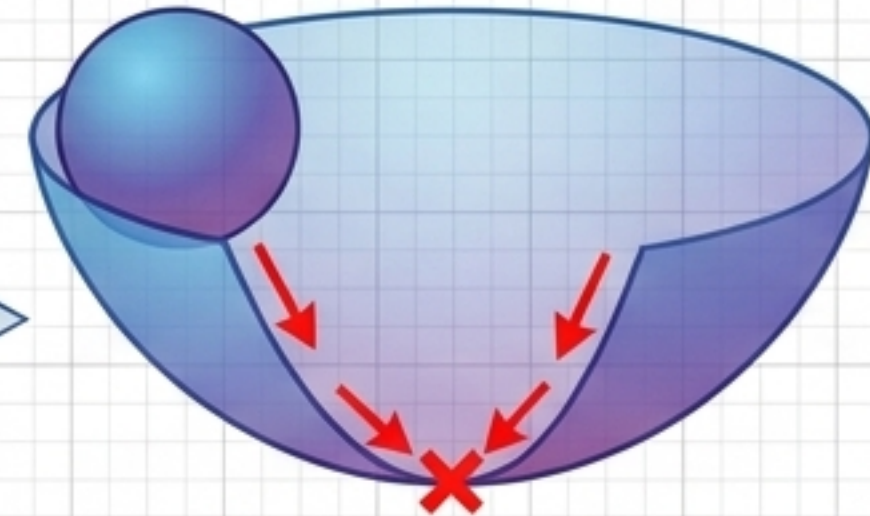
The mathematical decision to square the error does more than just penalize outliers—it defines the physical laws of the learning landscape.

$$(\dots)^2$$

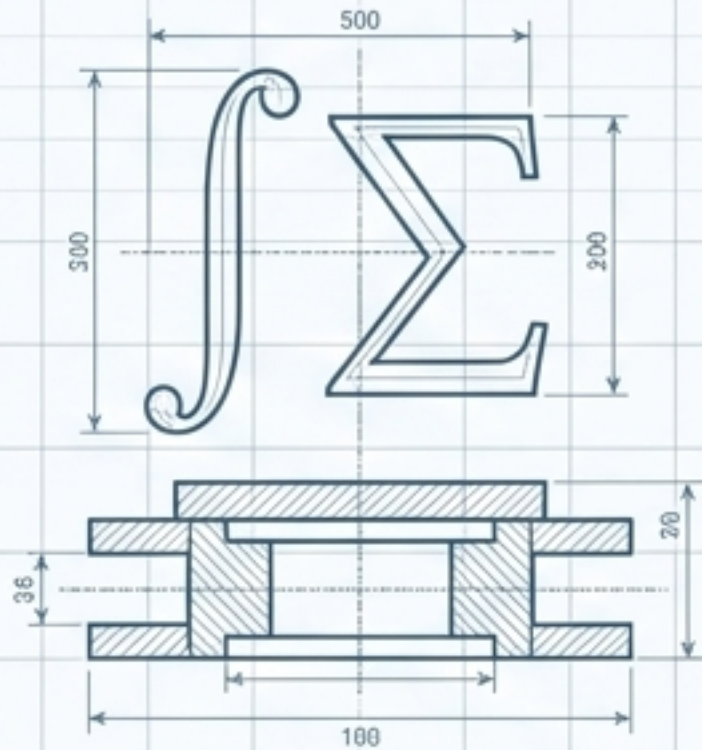


Why Gradient Descent Works

The squared-error cost function inherently generates a perfectly convex surface. Because there are no false valleys or local minima, following the downward slope from any random starting coordinate mathematically guarantees arrival at the single, true global optimum.

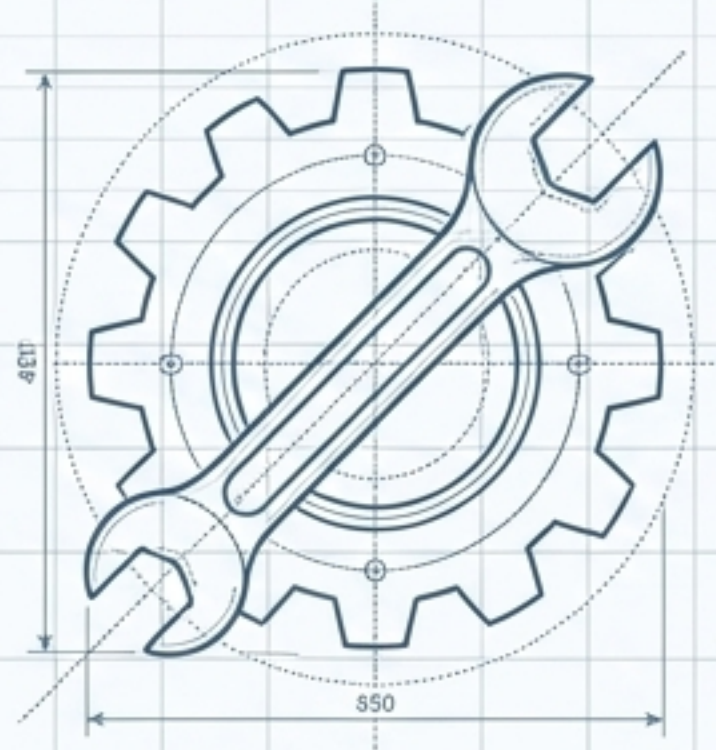


Core Engineering and Theoretical Takeaways



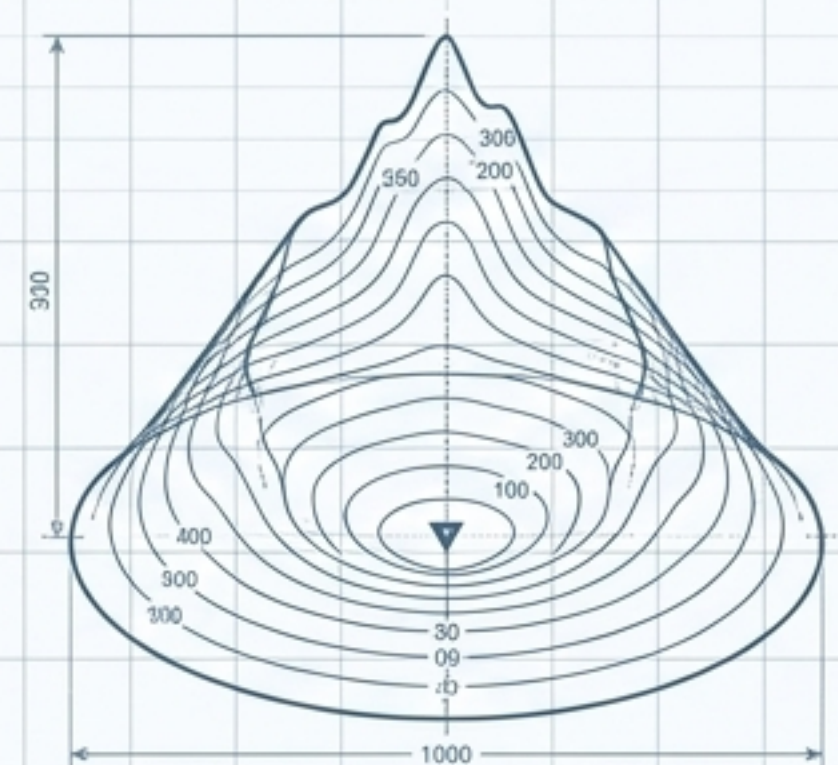
The Engine of Accuracy

The cost function $J(\mathbf{w}, \mathbf{b})$ is the definitive metric for model fit. Minimizing this single number perfectly aligns the linear prediction with the reality of the training data.



Engineered Resilience

Robust data science requires self-contained execution. Refactoring removed fragile dependencies and implemented graceful interactivity fallbacks, guaranteeing the code runs flawlessly across environments.



Theoretical Bedrock

The resulting convex topography—the smooth “soup bowl”—is the foundational requirement that allows gradient descent to converge reliably without getting trapped.